FICHE DE RÉVISION – HTML5, CSS & JAVASCRIPT

Format examen Commandes et syntaxes essentielles

1. HTML5 – Structure et balises

1.1. Structure de base d'une page HTML```html

```
<h1>Mon titre</h1>
<p>Mon paragraphe</p>

<script src="script.js"></script>
```

Balise	Rôle
`<!DOCTYPE html>`	Déclare HTML5
`<html>`	Racine du document
`<head>`	Informations sur la page
`<title>`	Titre de l'onglet
`<meta>`	Métadonnées
`<body>`	Contenu visible
`<h1>` à `<h6>`	Titres
`<p>`	Paragraphe
` `	Saut de ligne
`<hr>`	Ligne horizontale
`<!-- -->`	Commentaire

1.2. Titres et formatage du texte

```
<h1>Titre principal</h1>
<h2>Sous-titre</h2>
<p>Paragraphe normal</p>

<b>Texte en gras</b>
<strong>Texte important</strong>
<i>Texte italique</i>
<em>Texte accentué</em>
<small>Petit texte</small>
<mark>Texte marqué</mark>
<del>Texte supprimé</del>
<ins>Texte inséré</ins>
<sub>Indice</sub>
<sup>Exposant</sup>
```

À retenir :

- `<b>` : gras visuel.
- `<strong>` : importance sémantique.
- `<i>` : italique visuel.
- `<em>` : mise en valeur.
- `<sub>` : indice, exemple H_2O.
- `<sup>` : exposant, exemple X^2.

1.3. Liens et images

```
<!-- Lien -->
<a href="https://google.com">Google</a>

<!-- Ouvrir dans un nouvel onglet -->
<a href="page.html" target="_blank">Page</a>

<!-- Image -->
<img src="images/photo.jpg"
     alt="Description"
     width="300"
     height="200">
```

Attribut	Fonction
`href`	Destination du lien
`src`	Chemin du fichier
`alt`	Texte alternatif
`width`	Largeur
`height`	Hauteur
`target="_blank"`	Nouvel onglet
`target="_self"`	Même onglet

Chemins de fichiers :

```
<img src="image.jpg">              <!-- Même dossier -->
<img src="images/image.jpg">       <!-- Sous-dossier -->
<img src="../image.jpg">           <!-- Dossier parent -->
<a href="contact.html">Contact</a> <!-- Page locale -->
```

1.4. Listes

```
<!-- Liste non ordonnée -->
<ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ul>

<!-- Liste ordonnée -->
<ol>
    <li>Étape 1</li>
    <li>Étape 2</li>
</ol>
```

- `<ul>` : liste avec des puces.
- `<ol>` : liste numérotée.
- `<li>` : élément de liste.

1.5. Tableaux

```
<table border="1">
    <caption>Liste des étudiants</caption>

    <tr>
        <th>Nom</th>
        <th>Âge</th>
    </tr>

    <tr>
        <td>Ahmed</td>
        <td>20</td>
    </tr>

    <tr>
        <td>Ali</td>
        <td>21</td>
    </tr>
</table>
```

Balise / attribut	Fonction
`<table>`	Créer un tableau
`<tr>`	Créer une ligne
`<th>`	Cellule d'en-tête
`<td>`	Cellule normale
`<caption>`	Titre du tableau
`colspan="2"`	Fusionner 2 colonnes
`rowspan="2"`	Fusionner 2 lignes

Exemple de fusion :

```
<td colspan="2">Cellule fusionnée</td>

<td rowspan="2">Cellule fusionnée</td>
```

1.6. Block et Inline

Block	Inline
Commence sur une nouvelle ligne	Ne commence pas sur une nouvelle ligne
Occupe la largeur disponible	Occupe seulement l'espace nécessaire
`<div>`, `<p>`, `<h1>`	`<span>`, `<a>`, `<b>`

```
<div>Bloc 1</div>
<div>Bloc 2</div>

<p>Bonjour <span>tout le monde</span></p>
```

1.7. Class et ID

```
<p class="red">Texte 1</p>
<p class="red">Texte 2</p>

<h1 id="title">Titre unique</h1>
```

- `class` : peut être utilisée par plusieurs éléments.
- `id` : identifiant unique dans la page.

1.8. Formulaires

```
<form action="traitement.php" method="post">

    <label for="name">Nom :</label>
    <input type="text" id="name" name="name"
           placeholder="Votre nom" required>

    <br>

    <label for="password">Mot de passe :</label>
    <input type="password" id="password" name="password">

    <br>

    <input type="radio" name="gender" value="M"> Homme
    <input type="radio" name="gender" value="F"> Femme

    <br>

    <input type="checkbox" name="html"> HTML
    <input type="checkbox" name="css"> CSS

    <br>

    <input type="date" name="date">

    <br>

    <input type="submit" value="Envoyer">

</form>
```

Les différents types de `input` :

Syntaxe	Fonction
`type="text"`	Champ texte
`type="password"`	Mot de passe
`type="radio"`	Un seul choix
`type="checkbox"`	Plusieurs choix
`type="date"`	Date
`type="email"`	Adresse e-mail
`type="number"`	Nombre
`type="submit"`	Envoyer le formulaire
`type="reset"`	Réinitialiser

Attributs importants :

Attribut	Fonction
`name`	Nom du champ
`id`	Identifiant
`value`	Valeur
`placeholder`	Indication dans le champ
`required`	Champ obligatoire
`size`	Largeur visible en caractères
`pattern`	Expression régulière de validation
`list`	Référence à un `<datalist>`
`action`	Destination du formulaire
`method`	Méthode d'envoi

Pour lier un `label` à un `input`, il faut :

```
<label for="email">Email</label>
<input type="email" id="email">
```

Attention : les `radio buttons` ayant le même `name` permettent un seul choix. Les `checkboxes` permettent plusieurs choix.

1.9. Éléments sémantiques

Balise	Fonction
`<header>`	En-tête
`<nav>`	Navigation
`<main>`	Contenu principal
`<section>`	Section
`<article>`	Article indépendant
`<aside>`	Contenu secondaire
`<footer>`	Pied de page
`<figure>`	Contenu illustratif
`<figcaption>`	Légende
`<details>`	Informations supplémentaires
`<summary>`	Résumé de `<details>`

1.10. Audio, vidéo et iframe

```
<audio controls>
    <source src="audio.mp3" type="audio/mpeg">
</audio>

<video width="400" controls>
    <source src="video.mp4" type="video/mp4">
</video>

<iframe src="https://www.youtube.com/embed/VIDEO_ID"
        width="560"
        height="315">
</iframe>
```

- `<audio>` : intégrer un fichier audio.
- `<video>` : intégrer une vidéo.
- `<source>` : indiquer le fichier multimédia.
- `<iframe>` : intégrer une autre page ou une vidéo.

1.11. Responsive Web Design

Dans le `<head>` :

```
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">
```

Image responsive :

```
img {
    width: 100%;
    height: auto;
}
```

Taille du texte responsive :

```
h1 {
    font-size: 5vw;
}
```

---

2. CSS – Mise en forme et design

2.1. Les trois méthodes CSS

1. Inline CSS

```
<p style="color: red;">Bonjour</p>
```

2. Internal CSS

```
<head>
<style>
    p {
        color: red;
    }
</style>
</head>
```

3. External CSS

Dans HTML :

```
<link rel="stylesheet" href="style.css">
```

Dans `style.css` :

```
p {
    color: red;
}
```

2.2. Syntaxe CSS

```
selector {
    property: value;
}
```

Exemple :

```
p {
    color: red;
    text-align: center;
}
```

- `p` : selector.
- `color` : property.
- `red` : value.
- `{ }` : bloc de déclaration.
- `;` : sépare les déclarations.

2.3. Les sélecteurs CSS

Sélecteur	Syntaxe	Fonction
Element	`p`	Tous les paragraphes
ID	`#title`	Élément avec cet ID
Class	`.red`	Tous les éléments de cette classe
Universal	`*`	Tous les éléments
Grouping	`h1, p`	Plusieurs sélecteurs

Exemples :

```
p {
    color: red;
}

#title {
    color: blue;
}

.red {
    color: red;
}

p.center {
    text-align: center;
}

h1, h2, p {
    font-family: Arial;
}

* {
    margin: 0;
}
```

Plusieurs classes :

```
<p class="center large">Bonjour</p>
```

```
.center {
    text-align: center;
}

.large {
    font-size: 30px;
}
```

2.4. Commentaires CSS

```
/* Ceci est un commentaire */
```

2.5. Background – Arrière-plan

```
body {
    background-color: lightblue;

    background-image: url("image.jpg");

    background-position: center;

    background-attachment: fixed;
}
```

Propriété	Fonction
`background-color`	Couleur de fond
`background-image`	Image de fond
`background-position`	Position de l'image
`background-attachment`	Fixe ou défile avec la page
`background-repeat`	Répétition de l'image
`opacity`	Transparence (0 à 1)

Exemple :

```
div {
    background: lightblue url("image.jpg") center fixed;
    opacity: 0.8;
}
```

2.6. Borders – Bordures

```
div {
    border-width: 2px;
    border-style: solid;
    border-color: red;

    border-radius: 10px;
}
```

Les styles de bordure :

Valeur	Signification
`solid`	Ligne continue
`dotted`	Pointillée
`dashed`	Tirets
`double`	Double ligne
`groove`	Rainurée
`ridge`	Relief
`inset`	Enfoncée
`outset`	Sortante
`none`	Aucune
`hidden`	Cachée

Shorthand :

```
div {
    border: 2px solid red;
}
```

Bordures individuelles :

```
div {
    border-top: 2px solid red;
    border-right: 2px dashed blue;
    border-bottom: 2px solid green;
    border-left: 2px dotted black;
}
```

2.7. Margin et Padding

```
div {
    margin: 20px;
    padding: 15px;
}
```

- `margin` : espace extérieur à la bordure.
- `padding` : espace intérieur entre le contenu et la bordure.

Les quatre côtés :

```
div {
    margin-top: 10px;
    margin-right: 20px;
    margin-bottom: 10px;
    margin-left: 20px;

    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 10px;
    padding-left: 20px;
}
```

Shorthand :

```
/* Les 4 côtés */
margin: 10px;

/* Haut/Bas - Droite/Gauche */
margin: 10px 20px;

/* Haut - Droite - Bas - Gauche */
margin: 10px 20px 30px 40px;
```

Centrer horizontalement un élément de largeur définie :

```
div {
    width: 300px;
    margin: auto;
}
```

2.8. Dimensions

```
div {
    width: 300px;
    height: 200px;

    max-width: 500px;
    min-width: 200px;

    max-height: 400px;
    min-height: 100px;
}
```

2.9. Text – Propriétés du texte

```
p {
    color: red;

    text-align: center;

    text-decoration: none;

    text-transform: uppercase;

    text-indent: 30px;

    letter-spacing: 2px;

    word-spacing: 5px;

    line-height: 1.5;

    text-shadow: 2px 2px 5px gray;
}
```

Propriété	Fonction
`color`	Couleur du texte
`text-align`	Alignement
`text-decoration`	Décoration
`text-transform`	Majuscules/minuscules
`text-indent`	Retrait de la première ligne
`letter-spacing`	Espace entre lettres
`word-spacing`	Espace entre mots
`line-height`	Espace entre lignes
`text-shadow`	Ombre du texte

Valeurs courantes :

```
text-align: left;
text-align: right;
text-align: center;
text-align: justify;

text-decoration: underline;
text-decoration: overline;
text-decoration: line-through;
text-decoration: none;

text-transform: uppercase;
text-transform: lowercase;
text-transform: capitalize;
```

2.10. Fonts – Polices

```
p {
    font-family: Arial, sans-serif;

    font-size: 20px;

    font-style: italic;

    font-weight: bold;

    font-variant: small-caps;
}
```

Propriété	Fonction
`font-family`	Famille de police
`font-size`	Taille
`font-style`	Style
`font-weight`	Épaisseur
`font-variant`	Petites majuscules

Valeurs :

```
font-style: normal;
font-style: italic;
font-style: oblique;

font-weight: normal;
font-weight: bold;
font-weight: 100;
font-weight: 400;
font-weight: 700;
```

Shorthand :

```
p {
    font: italic bold 20px Arial;
}
```

2.11. CSS Links

```
a:link {
    color: blue;
}

a:visited {
    color: purple;
}

a:hover {
    color: red;
}

a:active {
    color: green;
}
```

État	Fonction
`:link`	Lien non visité
`:visited`	Lien déjà visité
`:hover`	Souris au-dessus
`:active`	Lien en cours de clic

2.12. CSS Lists

```
ul {
    list-style-type: square;
}

ol {
    list-style-type: upper-roman;
}
```

Types courants :

```
list-style-type: disc;
list-style-type: circle;
list-style-type: square;
list-style-type: none;

list-style-type: decimal;
list-style-type: upper-roman;
list-style-type: lower-roman;
list-style-type: upper-alpha;
list-style-type: lower-alpha;
```

Image comme marqueur :

```
ul {
    list-style-image: url("icon.png");
}
```

2.13. Display

```
div {
    display: block;
}

span {
    display: inline;
}

li {
    display: inline-block;
}

.hidden {
    display: none;
}
```

Valeur	Fonction
`block`	Nouvelle ligne, largeur disponible
`inline`	Même ligne, largeur nécessaire
`inline-block`	Même ligne avec dimensions configurables
`none`	Masquer l'élément

2.14. Position

```
div {
    position: relative;
    top: 10px;
    left: 20px;
}
```

Valeur	Fonction
`static`	Position normale par défaut
`relative`	Par rapport à sa position normale
`fixed`	Par rapport à la fenêtre, reste fixe
`absolute`	Par rapport au plus proche ancêtre positionné
`sticky`	Devient fixe selon le défilement

Propriétés de position :

```
top: 10px;
right: 10px;
bottom: 10px;
left: 10px;
```

2.15. z-index

Contrôle l'ordre de superposition des éléments.

```
.box1 {
    position: absolute;
    z-index: 1;
}

.box2 {
    position: absolute;
    z-index: 2;
}
```

Un `z-index` plus élevé place généralement l'élément devant un autre dans le même contexte d'empilement.

2.16. Overflow

```
div {
    overflow: auto;
}
```

Valeur	Fonction
`visible`	Contenu visible en dehors du bloc
`hidden`	Contenu coupé
`scroll`	Barres de défilement
`auto`	Barres si nécessaire

2.17. Float et Clear

```
img {
    float: left;
}
```

Valeur	Fonction
`float: left`	Flotte à gauche
`float: right`	Flotte à droite
`float: none`	Pas de flottement
`clear: left`	Évite le flottement à gauche
`clear: right`	Évite le flottement à droite
`clear: both`	Évite les flottements des deux côtés

Exemple :

```
img {
    float: left;
    margin-right: 15px;
}

p {
    clear: both;
}
```

2.18. Specificity – Spécificité

La spécificité détermine quelle règle CSS gagne lorsque plusieurs règles s'appliquent au même élément.

Sélecteur	Score du cours
`*`	0
`p`	1
`.class`	10
`:hover`	10
`#id`	100
`style=""`	1000
`!important`	10000

Exemple :

```
p {
    color: red;
}

.text {
    color: blue;
}

#title {
    color: green;
}
```

Si un élément possède simultanément un sélecteur `p`, une classe `.text` et un ID `#title`, la règle ID a la spécificité la plus élevée.

```
p {
    color: red !important;
}
```

`!important` force la priorité de cette déclaration dans les conditions normales de cascade.

---

3. JavaScript – Syntaxe et programmation

3.1. Introduction et syntaxe

JavaScript permet de rendre les pages Web interactives : boutons, animations, menus, etc.

Le JavaScript est sensible à la casse (*case-sensitive*).

```
let name = "Ahmed";
let Name = "Ali";
```

`name` et `Name` sont deux variables différentes.

Commentaires :

```
// Commentaire sur une ligne

/*
   Commentaire
   sur plusieurs lignes
*/
```

3.2. Variables : var, let et const

```
var x = 10;
let y = 20;
const z = 30;
```

Mot-clé	Fonction
`var`	Déclare une variable
`let`	Variable limitée au block
`const`	Constante, réaffectation interdite

Exemple :

```
let age = 20;
age = 21;       // Autorisé

const PI = 3.14;
// PI = 4;      // Erreur
```

Règles des noms de variables :

- Commencer par une lettre, `_` ou `$`.
- Ne pas commencer par un chiffre.
- Respecter les majuscules et minuscules.
- Ne pas utiliser un mot réservé.

3.3. Scope – Portée des variables

```
let global = 10;

function test() {
    let local = 20;
}
```

- Variable globale : déclarée en dehors d'une fonction.
- Variable locale : déclarée à l'intérieur d'une fonction.
- `let` et `const` ont une portée limitée au block `{ }`.
- `var` n'est pas limitée au block, mais à la fonction lorsqu'elle est déclarée dans une fonction.

3.4. Valeurs spéciales

Valeur	Signification
`undefined`	Variable déclarée sans valeur
`null`	Absence volontaire de valeur
`NaN`	Not a Number

```
let x;
console.log(x); // undefined

let y = null;
```

Accéder à une variable non déclarée provoque une `ReferenceError`.

3.5. Data Types – Types de données

JavaScript possède sept types primitifs et le type Object.

Type	Exemple
Boolean	`true`, `false`
null	`null`
undefined	`undefined`
Number	`10`, `3.14`
BigInt	`9007199254740992n`
String	`"Hello"`
Symbol	`Symbol("id")`
Object	`{name: "Ali"}`

3.6. Conversion des types

JavaScript est un langage à typage dynamique.

```
let x = 10;
x = "Hello";
```

Une même variable peut recevoir différents types de valeurs.

Attention à l'opérateur `+` :

```
console.log(10 + 5);   // 15
console.log("10" + 5); // "105"
console.log("10" - 5); // 5
console.log("10" * 2); // 20
```

Avec `+`, si une chaîne intervient, la concaténation peut être effectuée. Les autres opérateurs arithmétiques peuvent convertir les chaînes numériques en nombres.

3.7. Literals – Littéraux

Les littéraux représentent des valeurs fixes écrites directement dans le code.

String :

```
let name = "Ahmed";
let city = 'Agadir';
```

Number :

```
let age = 20;
let price = 15.5;
```

Boolean :

```
let isStudent = true;
```

Array :

```
let fruits = ["Apple", "Banana", "Orange"];
```

Object :

```
let student = {
    name: "Ahmed",
    age: 20
};
```

3.8. Arrays – Tableaux

```
let numbers = [10, 20, 30, 40];

console.log(numbers[0]); // 10
console.log(numbers[1]); // 20

console.log(numbers.length); // 4
```

Les indices commencent à `0`.

```
numbers[0] = 100;  // Modifier
numbers.push(50);  // Ajouter à la fin
numbers.pop();     // Supprimer le dernier élément
```

3.9. String – Chaînes de caractères

```
let text = "JavaScript";

console.log(text.length);
```

`length` donne le nombre de caractères.

Pour accéder à un caractère :

```
console.log(text[0]); // J
```

3.10. Instructions de bloc

Un block permet de regrouper plusieurs instructions entre accolades.

```
{
    let x = 10;
    let y = 20;

    console.log(x + y);
}
```

3.11. Conditions – if / else

```
let age = 20;

if (age >= 18) {
    console.log("Majeur");
}
else {
    console.log("Mineur");
}
```

if / else if / else :

```
let note = 15;

if (note >= 16) {
    console.log("Très bien");
}
else if (note >= 10) {
    console.log("Admis");
}
else {
    console.log("Échec");
}
```

3.12. Switch

Utilisé pour comparer une expression avec plusieurs valeurs possibles.

```
let day = 2;

switch (day) {

    case 1:
        console.log("Lundi");
        break;

    case 2:
        console.log("Mardi");
        break;

    case 3:
        console.log("Mercredi");
        break;

    default:
        console.log("Jour inconnu");
}
```

- `case` : valeur à comparer.
- `break` : sortir du `switch`.
- `default` : cas par défaut.

Attention : sans `break`, l'exécution continue dans les cases suivantes (*fall-through*).

3.13. Boucle for

Utilisée lorsque le nombre de répétitions est connu.

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```

Résultat :

```
0
1
2
3
4
```

Structure :

```
for (initialisation; condition; incrémentation) {
    // Instructions
}
```

3.14. Boucle while

Répète les instructions tant que la condition est vraie.

```
let i = 0;

while (i < 5) {
    console.log(i);
    i++;
}
```

3.15. Boucle do...while

La boucle s'exécute au moins une fois avant de vérifier la condition.

```
let i = 0;

do {
    console.log(i);
    i++;
}
while (i < 5);
```

Boucle	Particularité
`for`	Initialisation, condition et incrémentation dans une seule structure
`while`	Vérifie la condition avant l'exécution
`do...while`	Exécute une première fois avant de vérifier

3.16. Incrémentation et décrémentation

```
let x = 5;

x++;  // x = x + 1
x--;  // x = x - 1

x += 2; // x = x + 2
x -= 2; // x = x - 2
x *= 2; // x = x * 2
x /= 2; // x = x / 2
```

3.17. Fonctions

Une fonction est un ensemble d'instructions qui réalise une tâche.

Déclaration :

```
function sayHello() {
    console.log("Hello");
}
```

Appel :

```
sayHello();
```

Fonction avec paramètres :

```
function sum(a, b) {
    console.log(a + b);
}

sum(10, 20);
```

Fonction avec return :

```
function sum(a, b) {
    return a + b;
}

let result = sum(10, 20);

console.log(result); // 30
```

- `function` : déclarer une fonction.
- `sum` : nom de la fonction.
- `a`, `b` : paramètres.
- `return` : renvoyer un résultat.
- `sum(10, 20)` : appel de la fonction avec des arguments.

3.18. Math.random() et Math.floor()

Très importants pour les exercices.

```
let random = Math.random();
```

Génère un nombre aléatoire entre 0 inclus et 1 exclu.

```
let number = Math.floor(Math.random() * 10);
```

Génère un entier entre 0 et 9.

Générer un entier entre 1 et 100 :

```
let number = Math.floor(Math.random() * 100) + 1;
```

3.19. prompt() et alert()

```
let name = prompt("Enter your name");

alert("Hello " + name);
```

- `prompt()` : demander une valeur à l'utilisateur.
- `alert()` : afficher un message.

Pour convertir une saisie en nombre :

```
let age = Number(prompt("Enter your age"));
```

3.20. try...catch

Permet de gérer les exceptions.

```
try {
    let result = x;
}
catch (error) {
    console.log("Une erreur est survenue");
}
```

- `try` : instructions à essayer.
- `catch` : instructions exécutées lorsqu'une exception est levée.

---

4. Modèles complets à connaître pour l'examen

4.1. Exemple : formulaire HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Formulaire</title>
</head>
<body>

    <h1>Inscription</h1>

    <form>
        <label for="name">Nom :</label>
        <input type="text" id="name" required>

        <br><br>

        <label for="email">Email :</label>
        <input type="email" id="email" required>

        <br><br>

        <label for="password">Password :</label>
        <input type="password" id="password">

        <br><br>

        <input type="radio" name="gender" value="M"> Homme
        <input type="radio" name="gender" value="F"> Femme

        <br><br>

        <input type="checkbox" name="terms" required>
        Accepter les conditions

        <br><br>

        <input type="submit" value="S'inscrire">
    </form>

</body>
</html>
```

4.2. Exemple : CSS complet

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    background-color: lightgray;
    font-family: Arial, sans-serif;
}

header {
    background-color: navy;
    color: white;
    text-align: center;
    padding: 20px;
}

h1 {
    font-size: 30px;
    text-transform: uppercase;
}

.container {
    width: 80%;
    margin: auto;
    padding: 20px;
    background-color: white;
    border: 1px solid black;
    border-radius: 10px;
}

a {
    color: blue;
    text-decoration: none;
}

a:hover {
    color: red;
}
```

4.3. Exemple : JavaScript – Nombre à deviner

Exercice présent dans ton support JavaScript.

```javascript
let randomNumber = Math.floor(Math.random() * 100) + 1;

let attempts = 10;
let guess;

while (attempts > 0) {

    guess = Number(prompt("Guess a number between 1 and 100"));

    if (guess === randomNumber) {
        alert("Congratulations! You won!");
        break;
```

```
    }

    else if (guess > randomNumber) {
        alert("Too high!");
    }

    else {
        alert("Too low!");
    }

    attempts--;

    alert("Attempts remaining: " + attempts);
}

if (attempts === 0 && guess !== randomNumber) {
    alert("Game over! The number was " + randomNumber);
}
```---

5. Tableau récapitulatif – Les commandes à mémoriser

Objectif	Commande
Créer un titre	`<h1>`
Créer un paragraphe	`<p>`
Ajouter un lien	`<a href="">`
Ajouter une image	`<img src="">`
Créer une liste	`<ul>`, `<ol>`, `<li>`
Créer un tableau	``, `<tr>`, `<th>`, `<td>`
Créer un formulaire	`<form>`
Champ texte	``
Champ password	``
Bouton radio	``
Checkbox	``
Bouton submit	``
Sélectionner une classe CSS	`.class`
Sélectionner un ID CSS	`#id`
Sélectionner tous les éléments	`*`
Couleur du texte	`color`
Couleur de fond	`background-color`
Taille du texte	`font-size`
Police	`font-family`
Alignement	`text-align`
Bordure	`border`
Marge extérieure	`margin`
Marge intérieure	`padding`
Positionnement	`position`
Affichage	`display`
Superposition	`z-index`
Débordement	`overflow`
Flottement	`float`
Déclaration variable	`let`, `var`, `const`
Condition	`if`, `else`
Plusieurs conditions	`else if`
Choix multiples	`switch`, `case`, `break`
Boucle	`for`, `while`, `do...while`
Fonction	`function`
Retourner une valeur	`return`
Afficher un message	`alert()`
Demander une saisie	`prompt()`

```
| Nombre aléatoire | `Math.random()` |
| Arrondir vers le bas | `Math.floor()` |
| Gérer une erreur | `try`, `catch` |

## 6. Les pièges classiques de l'examen

À ne pas confondre

1. `.` et `#` en CSS :
   - `.title` = class.
   - `#title` = id.

2. `margin` et `padding` :
   - `margin` = extérieur.
   - `padding` = intérieur.

3. `radio` et `checkbox` :
   - `radio` = un seul choix par groupe.
   - `checkbox` = plusieurs choix possibles.

4. `for` et `while` :
   - `for` = boucle avec initialisation, condition et incrémentation.
   - `while` = répétition tant que la condition est vraie.

5. `while` et `do...while` :
   - `while` vérifie avant d'exécuter.
   - `do...while` exécute au moins une fois.

6. `=` et `===` :
   - `=` = affectation.
   - `===` = comparaison stricte.

7. `href` et `src` :
   - `href` = destination d'un lien.
   - `src` = source d'une image ou d'un média.

8. `display: none` et `visibility: hidden` :
   - `display: none` = élément retiré de la mise en page.
   - `visibility: hidden` = élément invisible, mais espace conservé.

9. `break` et `return` :
   - `break` = sortir d'une boucle ou d'un `switch`.
   - `return` = terminer une fonction et éventuellement renvoyer une valeur.

10. `undefined` et `null` :
    - `undefined` = valeur non définie.
    - `null` = absence volontaire de valeur.

---

**Sources utilisées :** tes cinq supports de cours, sans ajout de chapitres extérieurs.

- HTML5 Partie 1 <FileCite ref_id="turn0file2"/
- HTML5 Partie 2 <FileCite ref_id="turn0file3"/
- CSS Partie 1 <FileCite ref_id="turn0file0"/
- CSS Partie 2 <FileCite ref_id="turn0file1"/
- JavaScript Partie 1 <FileCite ref_id="turn0file4"/

* Remarque :* cette fiche couvre les notions présentes dans les PDF. Elle ne prétend pas
couvrir JavaScript DOM ou les événements, qui ne figurent pas dans le support fourni.
```

> **Conseil :** garde cette fiche comme référence pendant tes révisions. Apprends surtout les syntaxes et entraîne-toi à les écrire sans regarder.
>
> Je peux également te préparer un examen blanc couvrant les trois matières, avec des exercices pratiques et un corrigé détaillé.